CS6103E Software Systems Laboratory

Monsoon Semester (2025–26)

1st Year M.Tech CSE/CS(IS)

Python Programming-Practice set 2

Objectives

- To understand and implement Python exception handling using `try`, `except`, `else`, and `finally` blocks.
- To develop skills in manipulating and processing strings effectively in Python.
- To visualize data using suitable libraries and interpret the results meaningfully.

Submission date: On or before 19.11.2025 (11:59 pm)
Mode of submission: as a single .py file named as per the format `P2 <FIRST NAME> <ROLLNO>.py` (e.g. `P2 saritha m240064CS.py`)

Problem Set 1

Define functions to catch and handle exceptions appropriately.

1. Take a URL as input from the user and validate whether it starts with either "`http://`" or "`https://`", and contains a domain name that ends with a valid extension like ".com", ".org", or ".in". Use exception handling in your code with the following blocks:
   - Use a `try` block to perform the validation checks.
   - Raise an exception if the URL format is incorrect or the extension is invalid.
   - Use an `except` block to catch the invalid format exception and print "Invalid URL".
   - Use an `else` block to print "Valid URL" if no exceptions occur.
   - Use a `finally` block to print a message indicating that URL validation is complete regardless of outcome.

2. Take the URL of a web file as input and attempt to access the file using `urllib.request`. Use a `try` block to open and read the file. If the file is successfully accessed, save its contents to `output.html` and display a portion of the saved data. If the file is not found or the domain is invalid, catch the corresponding exceptions and write the error message to `error_log.txt`. Use an `else` block for successful access and a `finally` block to indicate that the process is complete.

3. Create a dictionary with 5 student ID-number to test-score key-value pairs as follows:
{`101: 89.5, 102: 76.0, 103: 91.0, 104: 64.5, 105: 88.0`}.
The dictionary should hold a maximum of 5 students only. Ask the user for a student ID, check if it is an integer and present in the dictionary. If the dictionary exceeds 5 students, raise an exception and disallow adding more entries. Print the score if the student ID is found; otherwise, raise and catch exceptions properly. Finally, print the dictionary regardless of exceptions.

4. Take a string as input from the user, which should represent a mathematical expression (e.g., "5+3*2"). Validate that the string contains only digits and the basic operators (+, -, *, /). There will be no spaces and no brackets in the input string. If the input contains any invalid characters, raise a `TypeError`. If there is any division by zero, raise a `ZeroDivisionError`. If the expression is valid, evaluate and print the result using the `eval()` function.

Problem set 2

1. Given two strings, a source string `s` and a pattern string `p` that can include two special characters: `?` (matches any single character) and `*` (matches any sequence of characters, including an empty sequence). Determine if the pattern `p` matches the entire source string `s`.

Input:
First input: string `s` (source string)
Second input: string `p` (pattern string)

Output:
`True` if the pattern matches the string, `False` otherwise.

Example:

Sample Input and Output

Input:
Enter the source string: `abcdef`
Enter the pattern string: `a*d*f`

Output:
Pattern matches string: `True`

2. Given a string, write a program to compress it using counts of repeated characters. If the compressed string is not shorter than the original, return the original string. For example, the string `"aabcccccaaa"` would become `"a2b1c5a3"`.

Input:
Enter a string `s`.

Output:
Compressed version of the string

Example:

Sample Input and Output

Input:
Enter the string: `aabcccccaaa`

Output:
Compressed string: `a2b1c5a3`

Problem set 3

1) Visualizing Student Performance

The dataset `student-scores.csv` contains information about students, including their demographic details, study habits, and marks obtained in various subjects such as Mathematics, Physics, Chemistry, Biology, English, His-

tory, and Geography. Perform the following visualization tasks using Python libraries such as `matplotlib` or `seaborn`.

a) Plot a bar graph showing the average marks of male and female students across all subjects.
b) Draw a line graph showing the variation in average marks of all subjects for students with and without part-time jobs.
c) Create a histogram displaying the distribution of students based on their weekly self-study hours.
d) Plot a pie chart showing the percentage of students engaged in extracurricular activities.

2) Outlier Detection Using Z-Score

Outliers are extreme data points that differ significantly from the rest of the dataset. They can be identified using statistical methods such as the Z-score, calculated as:

$$Z = \frac{(X - \mu)}{\sigma}$$

where X is a data point, μ is the mean, and σ is the standard deviation.

(a) Compute the Z-score for the attribute `weekly_self_study_hours` and identify students whose Z-score values fall outside the range of ± 2.

(b) Display these outlier students along with their `id`, `first_name`, and `weekly_self_study_hours`.

(c) Plot the Gaussian (normal) distribution curve of the Z-scores computed for the attribute `weekly_self_study_hours`. On the same plot, clearly mark the mean (0σ) and the vertical boundary lines corresponding to $\pm 1\sigma$, $\pm 2\sigma$, and $\pm 3\sigma$. Highlight the outlier points (i.e., students whose Z-scores satisfy $|Z| > 2$) using a distinct color.